DATABASE MANAGEMENT SYSTEMS

ASSIGNMENT – CO2

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.

Assignment information	**Details**
Course	CSA05 – Database Management Systems
Case study	Relational Design and Validation for Courier Handover Completion
Nature	Individual assignment
Total marks	50
Student name	Jagarajupalli Bhanuprakash Reddy
Register number	192524171
Department / Institution	AI&DS/SIMATS
Faculty	Dr.Shankar Shanmuganathan
Submission date	04-11-2026

CONTENTS AND MARK DISTRIBUTION

Task	Topic	Marks
1	Alternative relational schemas	5
2	Functional dependencies	2
3	Closures, keys and minimal cover	6
4	Update, insertion and deletion anomalies	4
5	Normal forms through BCNF	3
6	Lossless join, dependency preservation, 4NF and 5NF	3
7	Relational algebra and calculus	4
8	SQL implementation and testing	4
9	Experimental results	3
10	Theory versus experiment	4
11	Final schema recommendation	4
12	Re-evaluation design challenge	8
TOTAL		50

INTRODUCTION

SwiftSpan Parcels is a fictional courier operation whose handover sheets combine sender identity, service catalogue information, coordinator assignment details and periodic completion observations. The central design issue is that facts with different determinants are stored together. As a result,

a change to one sender, service or coordinator may require many repeated edits.

This report develops three alternative relational designs, derives the functional dependencies from the stated business rules, proves candidate keys and a minimal cover, analyses anomalies and normal forms, and provides relational algebra, tuple relational calculus, domain relational calculus and MySQL-oriented implementation examples. It then proposes an indexed read projection for the changed publication workload.

OBJECTIVES

- Identify the 14 attributes and business rules governing SwiftSpan Parcels.
- Compare a flat baseline, a partial decomposition and a BCNF decomposition.
- Derive candidate keys, attribute closures and a minimal cover using Armstrong's axioms.
- Prove lossless reconstruction and discuss dependency preservation and higher normal forms.
- Implement repeatable SQL structures and define a fair experimental method.
- Recommend a design using correctness, update cost, measured local evidence and the revised workload.

CASE MODEL, ATTRIBUTES AND ASSUMPTIONS

The assignment scenario defines one observation by (SenderID, ServiceID, DispatchWave). Each actor has five required catalogue items observed in four periods. Thus the target populations correspond to 500, 5,000 and 30,000 actors for 10,000, 100,000 and 600,000 observation rows respectively.

Attribute	Meaning	Declared determinant
SenderID	Sender identifier	—
SenderName	Sender name	SenderID

RouteGroup	Sender route cohort	SenderID
ServiceID	Internal service identifier	—
ServiceCode	Unique service business code	ServiceID; alternate direction ServiceCode → ServiceID
ServiceName	Catalogue display name	ServiceID
VolumeWeight	Service volume/weight factor	ServiceID
CoordinatorID	Coordinator assigned to sender/service scope	SenderID, ServiceID
CoordinatorName	Coordinator name	CoordinatorID
Hub	Coordinator hub	CoordinatorID
DispatchWave	Dispatch period/wave	—
Completion	Integer completion score 0–100	SenderID, ServiceID, DispatchWave
ServiceBand	Policy band derived from Completion	Completion
RouteFactor	Policy factor derived from ServiceBand	ServiceBand

Explicit modelling assumptions

- Identifiers are stable, non-null business keys. SenderID, ServiceID, CoordinatorID and DispatchWave are represented as integers; ServiceCode is a unique character code.
- Each sender has one SenderName and one RouteGroup at a point in the model. Each ServiceID has one ServiceCode, ServiceName and VolumeWeight; ServiceCode is unique and identifies the same catalogue object.
- Each (SenderID, ServiceID) assignment has exactly one CoordinatorID. A CoordinatorID identifies one CoordinatorName and one Hub.
- Every observation is unique for (SenderID, ServiceID, DispatchWave). Completion is an integer in the inclusive range 0–100.

- The fictional policy is deterministic. For the demonstration only, ServiceBand is assigned as LOW for Completion 0–38, STANDARD for 39–69 and HIGH for 70–100. RouteFactor is assigned as 0.80, 1.00 and 1.25 for LOW, STANDARD and HIGH respectively. These thresholds/factors are explicit modelling choices because the brief names the dependency but does not supply the policy values.
- Seed data should be generated deterministically and identically for each design. Cohort patterns are test-data conveniences only; they do not establish additional functional dependencies.
- All benchmark numbers must be collected on the local machine. This report does not fabricate execution times, storage measurements or throughput claims.

TASK 1 – DESIGN ALTERNATIVE RELATIONAL SCHEMAS (5 MARKS)

Design A: Flat repeated-data baseline

SWIFTSPAN_FLAT(SenderID, SenderName, RouteGroup, ServiceID, ServiceCode, ServiceName, VolumeWeight, CoordinatorID, CoordinatorName, Hub, DispatchWave, Completion, ServiceBand, RouteFactor). Primary key: (SenderID, ServiceID, DispatchWave). ServiceCode is a candidate alternate identifier for the catalogue entity, but in the flat observation relation it is not a key for a full observation because sender and wave are still needed.

DESIGN A – FLAT OBSERVATION RELATION

SWIFTSPAN_FLAT		
PK	**SenderID**	Identifier of the sender
	SenderName	Name of the sender
	RouteGroup	Route group of the sender
PK	**ServiceID**	Identifier of the service
UQ	**ServiceCode**	Unique business code of the service
	ServiceName	Name of the service
	VolumeWeight	Volume or weight of the service
	CoordinatorID	Identifier of the assigned coordinator
	CoordinatorName	Name of the coordinator
	Hub	Operating hub of the coordinator
PK	**DispatchWave**	Dispatch period / wave number
	Completion	Percentage completion (0 – 100)
	ServiceBand	Band derived from Completion
	RouteFactor	Factor derived from ServiceBand

Logical Observation Key

(SendierID, ServiceID, DispatchWave)

Key Summary

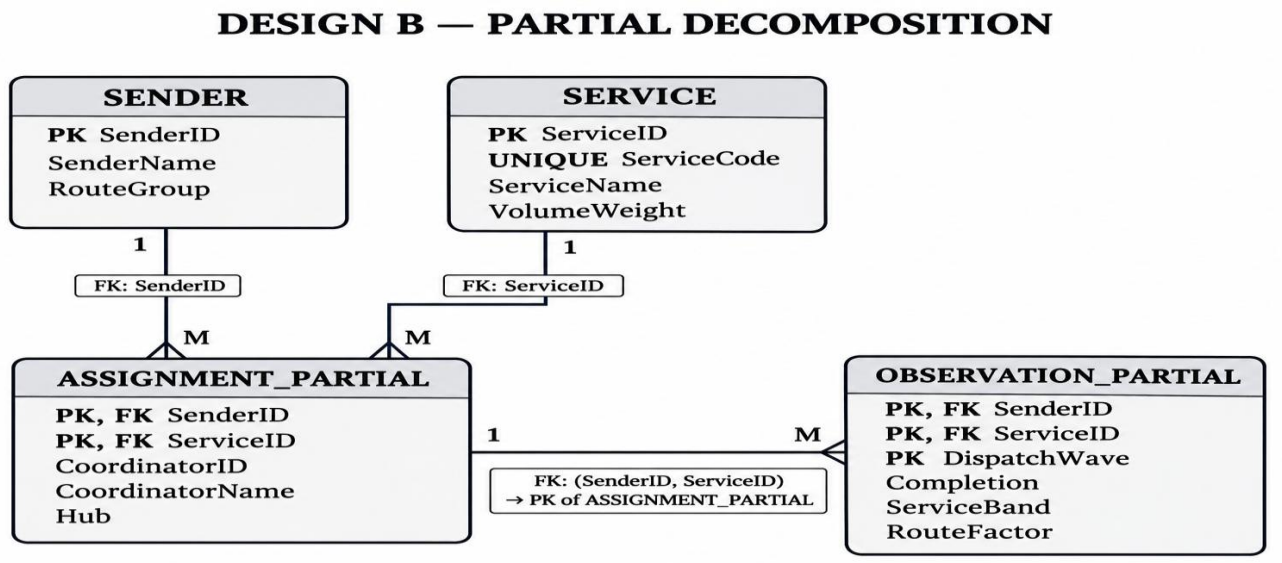
Primary Key : (SenderID, ServiceID, DispatchWave)

Candidate Key : {SenderID, ServiceCode, DispatchWave}

This design is retained as the baseline because it resembles the original handover sheet and provides a single-table observation lookup. It is expected to have the largest repetition and broadest update footprint.

Design B: Partial decomposition

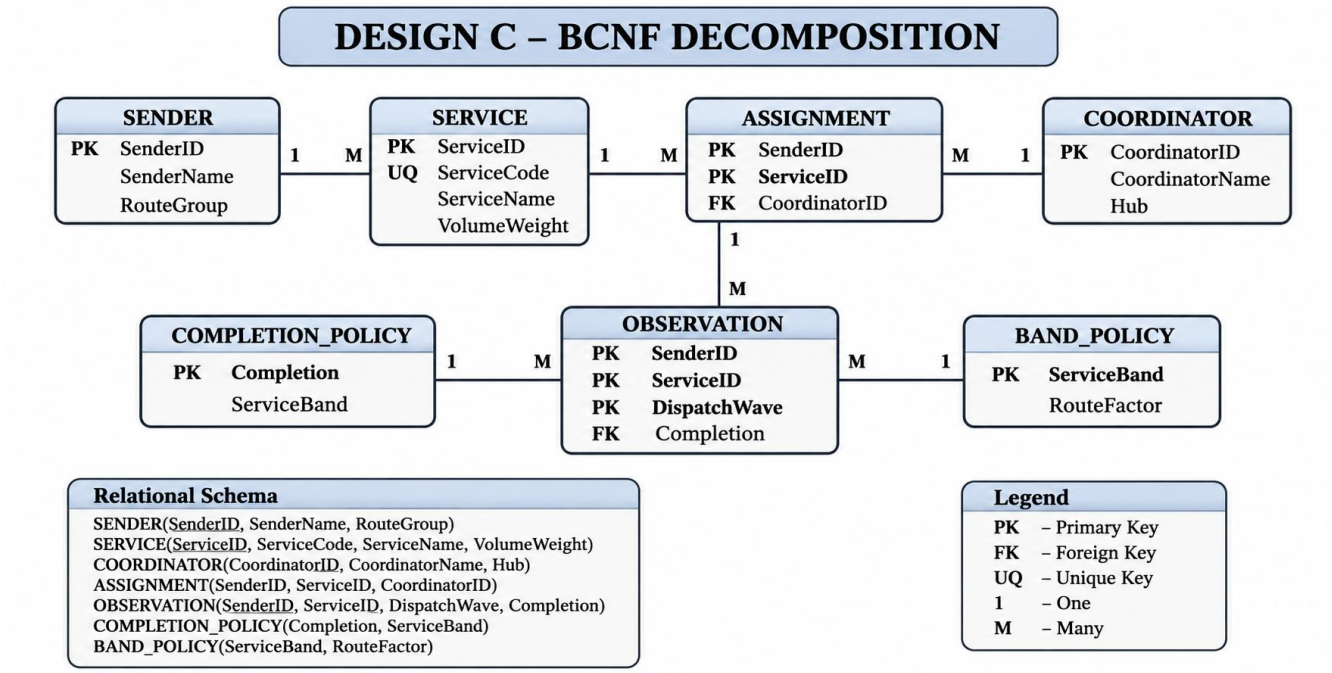
SENDER(SenderID PK, SenderName, RouteGroup); SERVICE(ServiceID PK, ServiceCode UNIQUE, ServiceName, VolumeWeight); ASSIGNMENT_PARTIAL(SenderID FK, ServiceID FK, CoordinatorID, CoordinatorName, Hub; PK(SenderID, ServiceID)); OBSERVATION_PARTIAL(SenderID, ServiceID, DispatchWave; PK(SenderID, ServiceID, DispatchWave), Completion, ServiceBand, RouteFactor).



This removes repeated sender identity and service catalogue descriptions from each observation. It deliberately retains CoordinatorName and Hub in the assignment row even though CoordinatorID determines them, and retains ServiceBand and RouteFactor in observation rows even though Completion and ServiceBand determine them. It is therefore a meaningful intermediate design, not a BCNF design.

Design C: BCNF decomposition

SENDER(SenderID PK, SenderName, RouteGroup); SERVICE(ServiceID PK, ServiceCode UNIQUE, ServiceName, VolumeWeight); COORDINATOR(CoordinatorID PK, CoordinatorName, Hub); ASSIGNMENT(SenderID FK, ServiceID FK, CoordinatorID FK; PK(SenderID, ServiceID)); OBSERVATION(SenderID, ServiceID, DispatchWave; PK(SenderID, ServiceID, DispatchWave), Completion); COMPLETION_POLICY(Completion PK, ServiceBand); BAND_POLICY(ServiceBand PK, RouteFactor).



The design places each fact with its determinant. The alternate key ServiceCode is enforced with UNIQUE in SERVICE. The observation row references the sender/service assignment scope and stores only the observed Completion value. Policy tables encode the fixed mappings once.

Design	**Strength**	**Trade-off / test purpose**
A — Flat	Simple single-table reads; useful baseline	Repeated facts and multi-row updates; measures anomaly cost.
B — Partial	Removes sender and service catalogue repetition	Coordinator identity and policy consequences remain duplicated.
C — BCNF	Separates every stated determinant; strong update integrity	More joins for full handover display; best normalized source of truth.

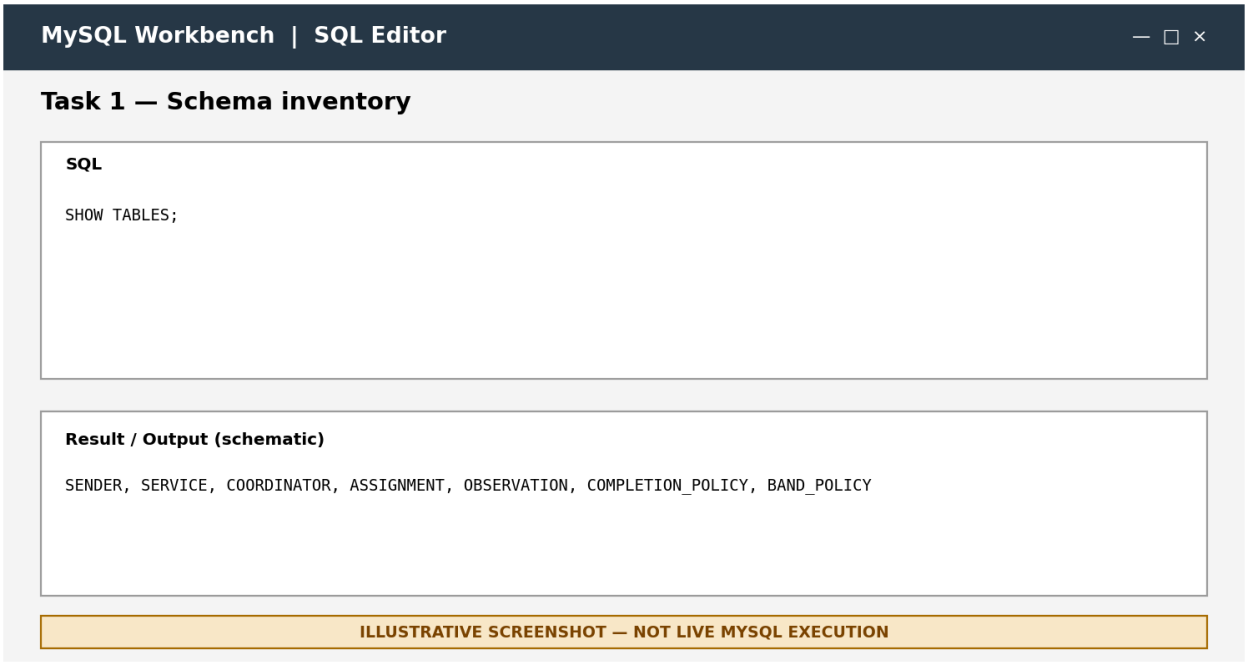


Figure 1. Illustrative schema inventory view; it represents the expected table inventory, not a live server capture.

TASK 2 – IDENTIFY THE FUNCTIONAL DEPENDENCIES (2 MARKS)

A functional dependency X → Y means that any two valid rows agreeing on X must also agree on Y. The dependencies below come from the scenario's business rules, not from accidental patterns in generated test data.

Functional dependency	Classification relative to observation key K	Reason
SenderID → SenderName, RouteGroup	Partial	Sender identity does not depend on service or wave.
ServiceID → ServiceCode, ServiceName, VolumeWeight	Partial	Catalogue facts depend on service identity.
ServiceCode → ServiceID	Partial / alternate identifier	ServiceCode is declared unique for a catalogue object.

(SenderID, ServiceID) → CoordinatorID	Partial	Assignment scope is fixed across dispatch waves.
CoordinatorID → CoordinatorName, Hub	Transitive	Coordinator details depend on CoordinatorID.
(SenderID, ServiceID, DispatchWave) → Completion	Full	Completion is recorded for one observation key.
Completion → ServiceBand	Transitive from K	Fixed policy maps completion to one band.
ServiceBand → RouteFactor	Transitive from K	Fixed policy maps band to one factor.

The consequence (SenderID, ServiceID) → CoordinatorName, Hub is implied by (SenderID, ServiceID) → CoordinatorID and CoordinatorID → CoordinatorName, Hub. It should not be repeated as an independent primitive rule.

False dependencies: a generated rule such as RouteGroup → SenderID may appear true if test data happen to assign one sender per group, but the scenario does not guarantee it. Likewise, a single DispatchWave in a small sample cannot establish that DispatchWave determines Completion or sender identity.

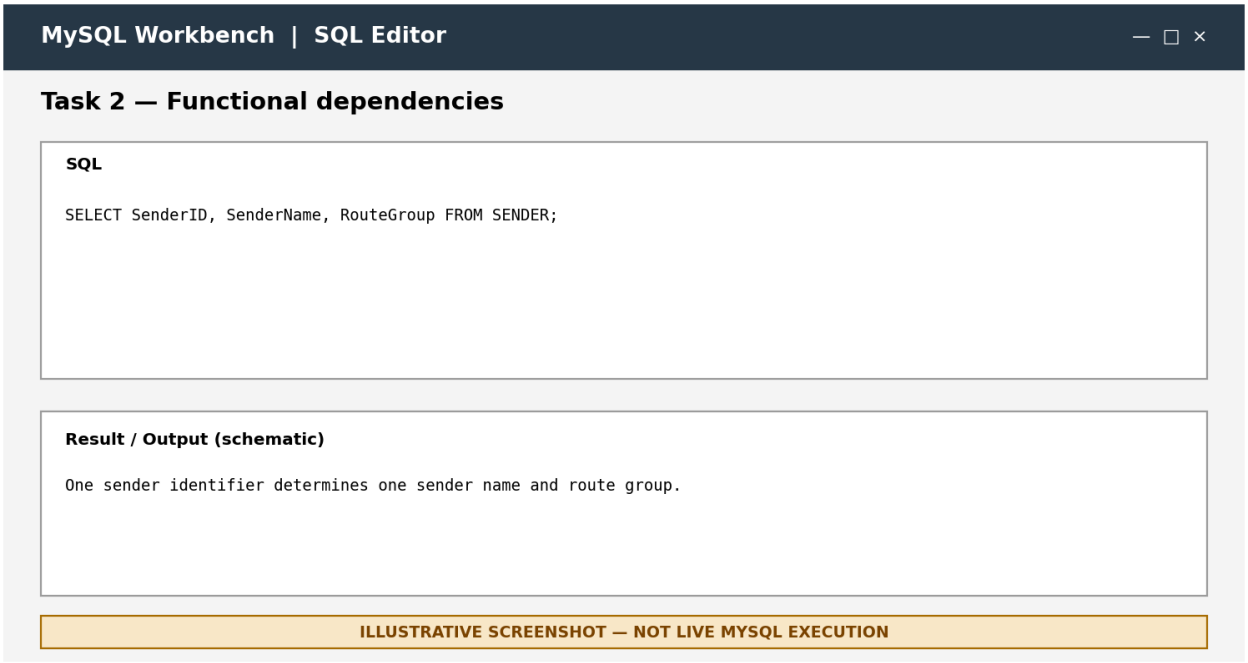


Figure 2. Illustrative dependency inspection. The result statement describes the rule being validated; it is not measured output.

TASK 3 – CLOSURES, KEYS AND MINIMAL COVER (6 MARKS)

3.1 Armstrong derivation: scope to coordinator name

(1) SenderID, ServiceID → CoordinatorID [given]
(2) CoordinatorID → CoordinatorName, Hub [given]
(3) SenderID, ServiceID → CoordinatorName, Hub [transitivity]

The scope determines the assigned coordinator. The coordinator identifier determines its name and hub. By transitivity, the assignment scope determines those details, but the minimal cover should retain the two primitive dependencies rather than the redundant consequence.

3.2 Closure of the observation key

K = {SenderID, ServiceID, DispatchWave}
K+ initially = {SenderID, ServiceID, DispatchWave}
SenderID → SenderName, RouteGroup
  add SenderName, RouteGroup
ServiceID → ServiceCode, ServiceName, VolumeWeight
  add ServiceCode, ServiceName, VolumeWeight

{SenderID, ServiceID} → CoordinatorID
  add CoordinatorID
CoordinatorID → CoordinatorName, Hub
  add CoordinatorName, Hub
K → Completion
  add Completion
Completion → ServiceBand
  add ServiceBand
ServiceBand → RouteFactor
  add RouteFactor
K+ = all 14 attributes. Therefore K is a superkey.

3.3 Closure using the alternate business identifier

K2 = {SenderID, ServiceCode, DispatchWave}
K2+ initially = {SenderID, ServiceCode, DispatchWave}
ServiceCode → ServiceID
  add ServiceID
Apply the dependencies from the previous closure:
  add SenderName, RouteGroup, ServiceName, VolumeWeight,
    CoordinatorID, CoordinatorName, Hub, Completion,
    ServiceBand, RouteFactor
K2+ = all 14 attributes. Therefore K2 is a superkey.

3.4 Candidate keys and minimality

The candidate keys are {SenderID, ServiceID, DispatchWave} and {SenderID, ServiceCode, DispatchWave}. They are minimal because no dependency determines SenderID or DispatchWave, so both must be present in every key. The service component must contain either ServiceID or ServiceCode: ServiceID determines ServiceCode and ServiceCode determines ServiceID. Removing any one member from either key leaves one of these indispensable dimensions missing.

Prime attributes are SenderID, DispatchWave, ServiceID and ServiceCode because each appears in at least one candidate key. The remaining ten attributes are non-prime.

3.5 Minimal cover

SenderID → SenderName
SenderID → RouteGroup

ServiceID → ServiceCode
ServiceID → ServiceName
ServiceID → VolumeWeight
ServiceCode → ServiceID
SenderID, ServiceID → CoordinatorID
CoordinatorID → CoordinatorName
CoordinatorID → Hub
SenderID, ServiceID, DispatchWave → Completion
Completion → ServiceBand
ServiceBand → RouteFactor

Each right-hand side is a singleton. No left-side attribute is extraneous: neither SenderID nor ServiceID can be removed from the assignment determinant, and none of the three observation-key attributes can be removed from the Completion determinant. The redundant scope-to-CoordinatorName/Hub dependency is omitted because it is implied by the two coordinator dependencies.

Equivalence check: every listed minimal-cover dependency is in or follows directly from the original business rules. Conversely, the original scope-to-name consequence follows by transitivity; grouped right-hand sides are reconstructed by union; therefore the reduced cover implies the original stated dependencies.

The assignment mentions an augmented dependency containing an extraneous VolumeWeight attribute but does not print that augmented expression in the supplied brief. In this solution, VolumeWeight remains a genuine catalogue attribute determined by ServiceID. It is not removed from the data model; only an attribute proven extraneous on a particular FD left/right side may be removed from that FD.

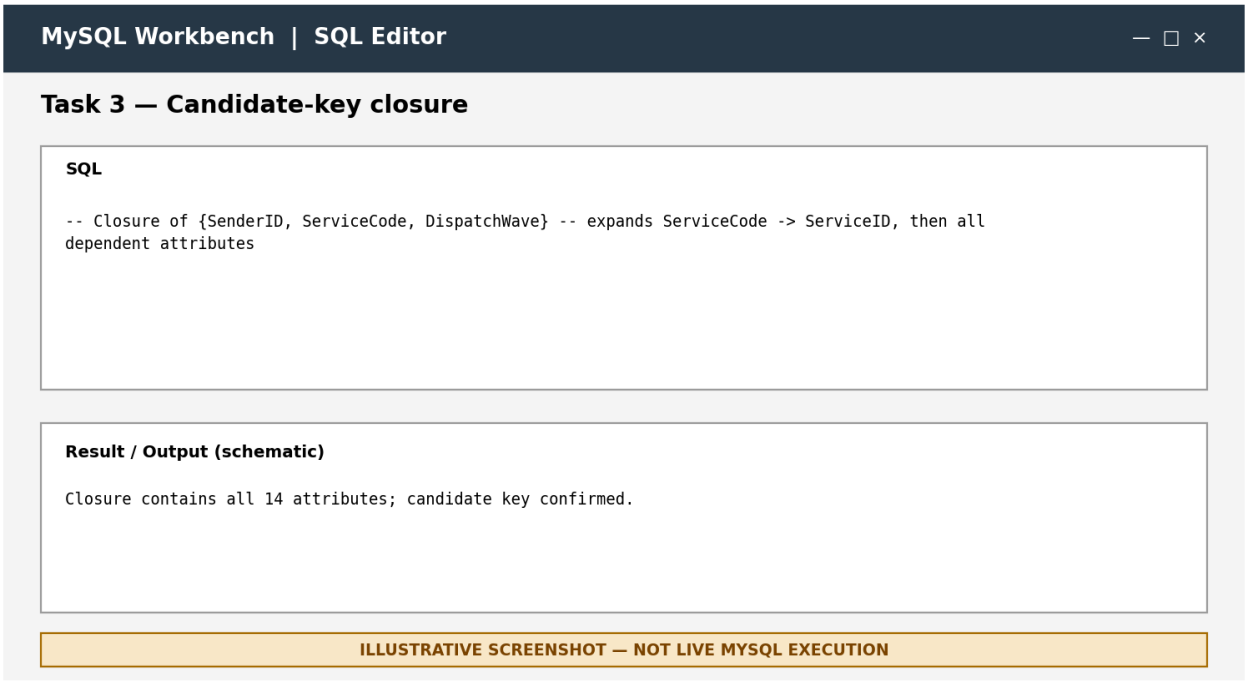


Figure 3. Illustrative closure output showing the alternate-key closure reaching the universal attribute set.

TASK 4 – UPDATE, INSERTION AND DELETION ANOMALIES (4 MARKS)

Insertion anomaly

In the flat design, a new service cannot be stored unless a sender observation row is also created, because every row requires the composite observation key. The normalized SERVICE table can hold a catalogue item before any sender is assigned to it.

Deletion anomaly

If the final observation for ServiceID=1 is deleted from SWIFTSPAN_FLAT, the only stored copy of that service's ServiceCode, ServiceName and VolumeWeight may disappear. In Design C, deleting an OBSERVATION row does not delete the SERVICE catalogue row.

Update anomaly

If ServiceName changes for ServiceID=1, the flat design must update every observation carrying ServiceID=1. If one row is missed, the same service identifier appears with inconsistent names. In Design B and C, the service name is stored once in SERVICE.

Observation rows	Actors (5 services × 4 waves)	Flat rows potentially updated for ServiceID=1	Normalized SERVICE rows updated
10,000	500	2,000	1
100,000	5,000	20,000	1
600,000	30,000	120,000	1

The counts above follow the assignment's fixed population structure: each actor contributes five services × four waves = 20 observations. Thus each service is represented by 4 observations per actor; across the actor population this gives 2,000, 20,000 and 120,000 rows for a selected service if service participation is balanced. This is a logical row-count estimate from the seed design, not a physical I/O measurement.

Design B still repeats CoordinatorName and Hub once per assignment scope if those values are retained there, and repeats ServiceBand/RouteFactor per observation. A change to coordinator name affects every assignment row carrying that CoordinatorID; a policy change affects every observation row carrying the affected band.

Storage comparison must distinguish estimates from measurements. Flat rows carry all 14 attributes; Design C stores sender, service, coordinator, assignment, observation and policy facts in separate relations. Actual allocated bytes depend on data types, indexes, page fill, row format and engine. Do not report estimated logical payload as measured InnoDB allocation.

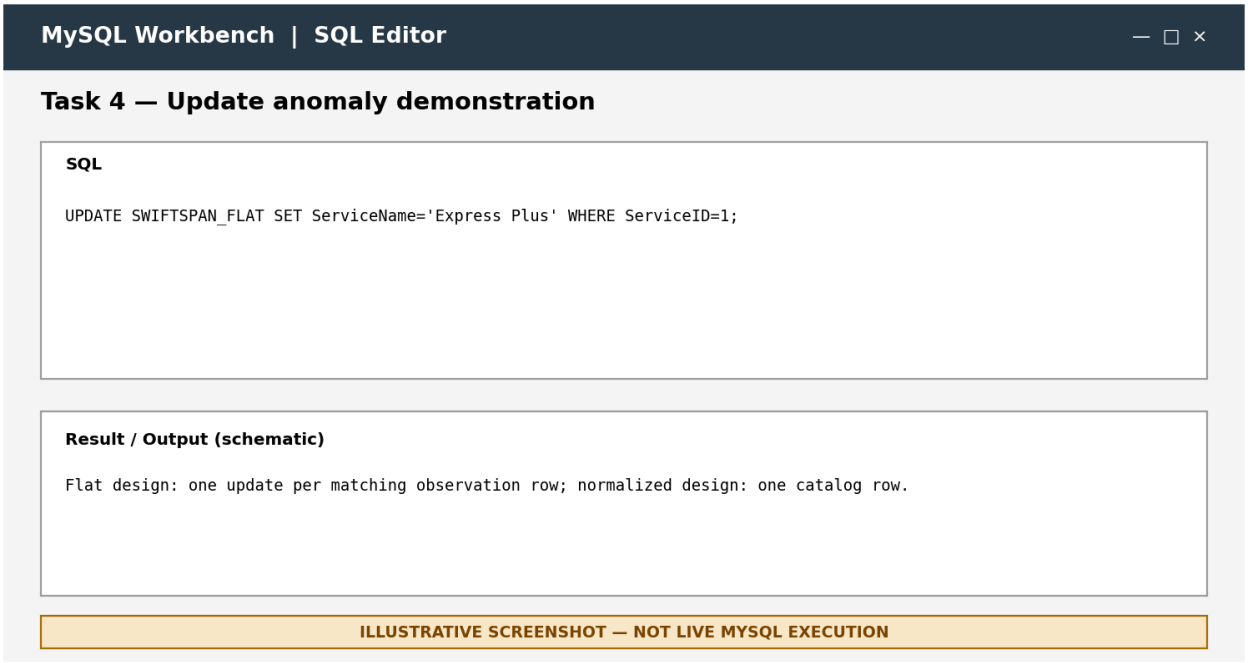


Figure 4. Illustrative update-anomaly screen. The row counts are derived from the stated population pattern, not from an executed UPDATE.

TASK 5 – NORMAL FORMS UP TO BCNF (3 MARKS)

1NF: every relation stores atomic values and has no repeating groups. The flat relation can be in 1NF if each cell is atomic, but that alone does not eliminate its partial and transitive dependencies.

2NF: a 1NF relation must have no non-prime attribute partially dependent on a proper subset of a candidate key. In the flat relation, SenderID → SenderName/RouteGroup and ServiceID → catalogue attributes are partial dependencies. The assignment scope also determines CoordinatorID. Therefore the flat relation violates 2NF.

3NF: for every non-trivial X → A, either X is a superkey or A is prime. The flat relation violates 3NF because SenderID → SenderName has a non-superkey determinant and a non-prime dependent. Design B still violates BCNF: CoordinatorID → CoordinatorName, Hub has a determinant that is not a superkey of ASSIGNMENT_PARTIAL, and Completion →

ServiceBand plus ServiceBand → RouteFactor violate BCNF in OBSERVATION_PARTIAL.

BCNF: every non-trivial determinant must be a superkey. Each relation in Design C has its determinant as a key: SenderID in SENDER; ServiceID and alternate ServiceCode in SERVICE; CoordinatorID in COORDINATOR; (SenderID, ServiceID) in ASSIGNMENT; the observation triple in OBSERVATION; Completion in COMPLETION_POLICY; ServiceBand in BAND_POLICY.

Dependency-preserving synthesis

The minimal-cover synthesis groups attributes by determinant: SENDER(SenderID, SenderName, RouteGroup); SERVICE(ServiceID, ServiceCode, ServiceName, VolumeWeight); COORDINATOR(CoordinatorID, CoordinatorName, Hub); ASSIGNMENT(SenderID, ServiceID, CoordinatorID); OBSERVATION(SenderID, ServiceID, DispatchWave, Completion); COMPLETION_POLICY(Completion, ServiceBand); BAND_POLICY(ServiceBand, RouteFactor). Remove any relation contained wholly in another; ensure a candidate-key relation is present. This grouping preserves the listed dependencies.

3NF but not BCNF example

Consider R(A,B,D) with FDs AB → D and D → B. Candidate keys are AB and AD. All attributes are prime (A, B, D), so D → B satisfies 3NF because B is prime, but violates BCNF because D is not a superkey. A 3NF synthesis may preserve D → B, whereas a BCNF decomposition can lose direct dependency preservation and require a join to check it.

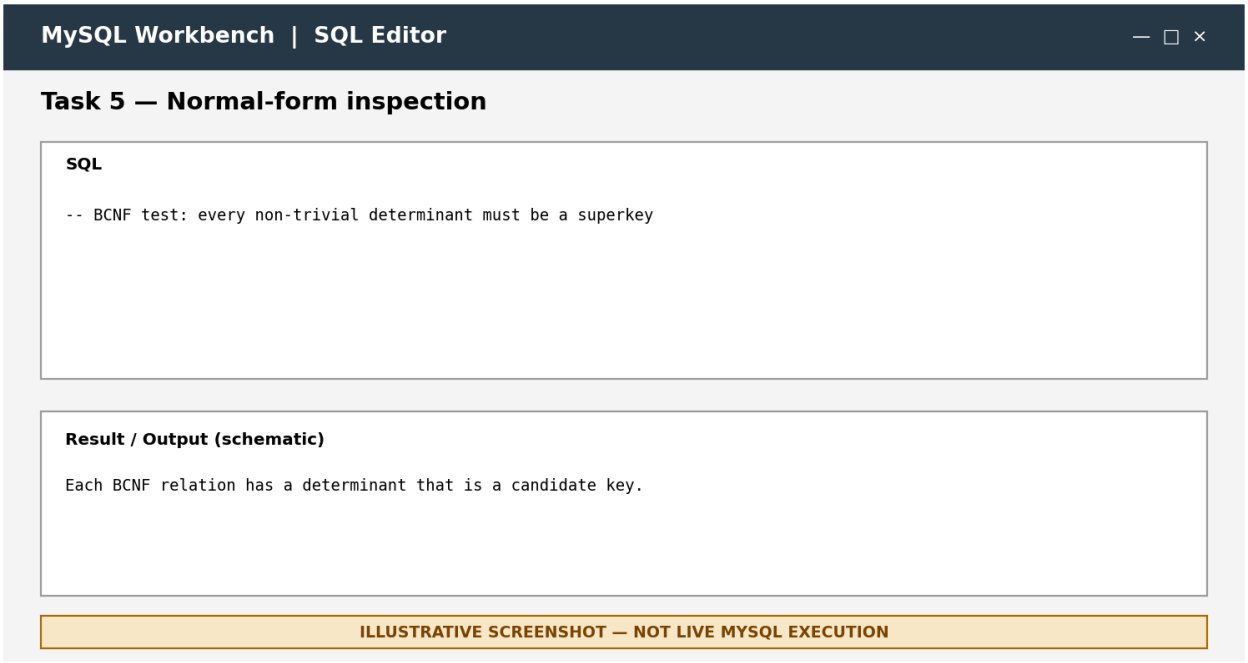


Figure 5. Illustrative normal-form test output. The classification is a dependency-based derivation.

TASK 6 – LOSSLESS JOIN, DEPENDENCY PRESERVATION AND HIGHER NORMAL FORMS (3 MARKS)

Lossless join argument

The BCNF design decomposes the universal relation along determinants that are keys of the projected relations. For a binary decomposition R → R1,R2, the common attributes must functionally determine all attributes of at least one side for a lossless join. The SENDER split uses SenderID → SenderName, RouteGroup; SERVICE uses ServiceID → catalogue attributes; COORDINATOR uses CoordinatorID → CoordinatorName, Hub; ASSIGNMENT uses (SenderID,ServiceID) → CoordinatorID; and policy splits use Completion → ServiceBand and ServiceBand → RouteFactor. These are determinant-based lossless splits.

A bad split that places SERVICE details in two fragments joined only on ServiceID is lossless only if the common ServiceID is a key on at least one

side. If the split instead uses a non-key common attribute such as ServiceName, two distinct services sharing a display name can create spurious combinations. Therefore joins must use declared identifiers, not descriptive labels.

Dependency preservation

The dependencies are locally enforceable in the BCNF relations: sender facts in SENDER; catalogue facts and alternate code uniqueness in SERVICE; assignment scope in ASSIGNMENT; coordinator facts in COORDINATOR; observation key in OBSERVATION; and policy mappings in their policy tables. The implied scope-to-coordinator-name rule is checked through the assignment/coordinator join rather than stored as a duplicate column.

4NF example: independent multivalued facts

Suppose a service supports several independent transport modes and several independent document types. For a given ServiceID, the sets are independent: ServiceID ↠ TransportMode and ServiceID ↠ DocumentType. Storing both in SERVICE_RESOURCE(ServiceID, TransportMode, DocumentType) creates a cross-product. Decompose into SERVICE_MODE(ServiceID, TransportMode) and SERVICE_DOCUMENT(ServiceID, DocumentType). Each relation is in 4NF if ServiceID is the determinant of its multivalued set and no non-trivial MVD has a non-superkey determinant.

5NF example: three-way authorization

Let APPROVAL(Operator, Item, Channel) represent allowed triples. If the business rule is that a valid triple is exactly the intersection of pairwise approvals, then the relation has a join dependency *{OperatorItem, OperatorChannel, ItemChannel}. Decompose into OPERATOR_ITEM(Operator,Item), OPERATOR_CHANNEL(Operator,Channel) and ITEM_CHANNEL(Item,Channel). This is a genuine 5NF case only if the

pairwise rule is a real business constraint; it must not be inferred from sample rows.

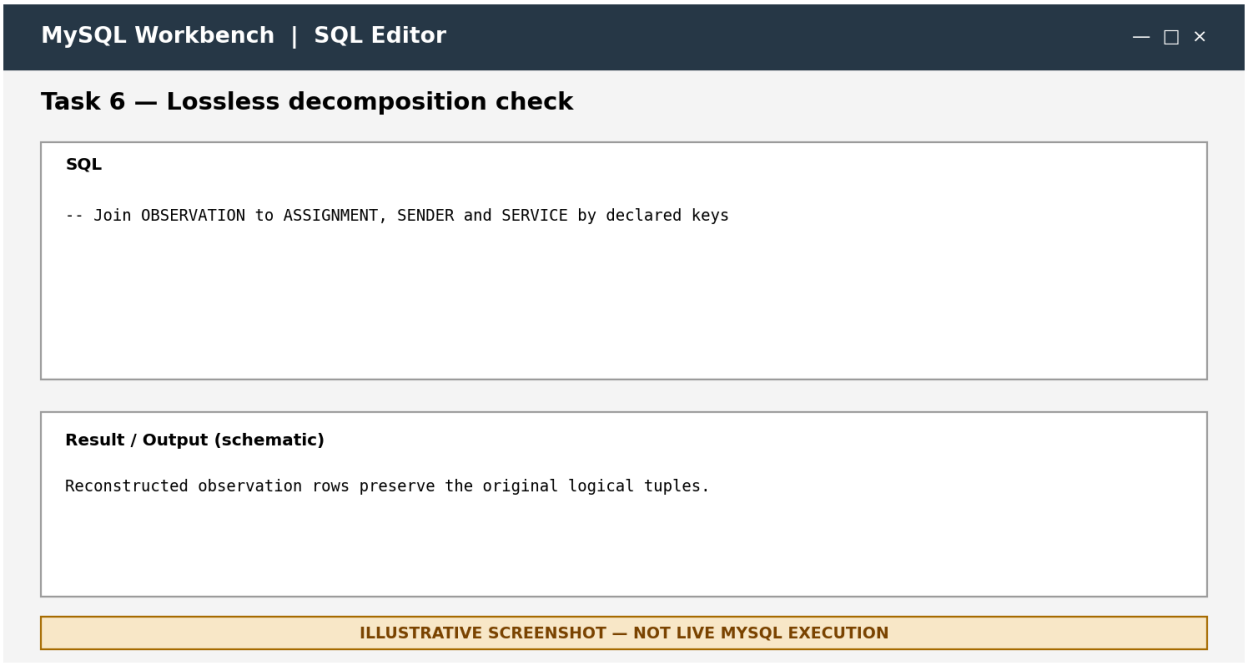


Figure 6. Illustrative lossless reconstruction output; the explanation relies on determinant-based joins.

TASK 7 – RELATIONAL ALGEBRA AND CALCULUS (4 MARKS)

Notation: S = SENDER, V = SERVICE, A = ASSIGNMENT, O = OBSERVATION, C = COORDINATOR. π is projection, σ selection, ⋈ join, γ grouping/aggregation, and − set difference. The adjusted index is defined here as Completion × VolumeWeight × RouteFactor. If the course's instructor supplied a different explicit formula elsewhere, substitute that exact formula.

Query 1: actor 1, wave 4, weighted adjusted index

RA: π SenderID, ServiceID, DispatchWave, Completion,
(Completion × VolumeWeight × RouteFactor)→AdjustedIndex
(σ SenderID=1 ∧ DispatchWave=4 (O ⋈ V ⋈ P ⋈ R))

TRC: { t | ∃o∈O ∃v∈V ∃p∈P ∃r∈R (o.SenderID=1 ∧ o.DispatchWave=4
∧ o.ServiceID=v.ServiceID ∧ o.Completion=p.Completion

∧ p.ServiceBand=r.ServiceBand ∧
t.AdjustedIndex=o.Completion*v.VolumeWeight*r.RouteFactor) }

The product is an extended relational-algebra expression because classical relational algebra has no arithmetic projection operator in its basic form.

Query 2: senders meeting 39 for every required service

RA: Required = π SenderID,ServiceID(ASSIGNMENT)
Good = π SenderID,ServiceID(σ Completion≥39(ASSIGNMENT ⋈ OBSERVATION))
MissingOrBad = Required − Good
BadSenders = π SenderID(MissingOrBad)
Answer = π SenderID(Required) − BadSenders
TRC: {s | SENDER(s) ∧ ¬∃a∈ASSIGNMENT [a.SenderID=s.SenderID ∧ ¬∃o∈OBSERVATION
(o.SenderID=a.SenderID ∧ o.ServiceID=a.ServiceID ∧ o.Completion≥39)]}

The TRC uses double negation to express “for every required service”. In an implementation with changing requirements, use the ASSIGNMENT relation as the authoritative set of required sender/service pairs.

Query 3: observed services with no value below 39 in wave 4

RA: Observed4 = π ServiceID(σ DispatchWave=4(O))
Bad4 = π ServiceID(σ DispatchWave=4 ∧ Completion<39(O))
Answer = Observed4 − Bad4
TRC: {v | SERVICE(v) ∧ ∃o∈O(o.ServiceID=v.ServiceID ∧ o.DispatchWave=4)
∧ ¬∃o2∈O(o2.ServiceID=v.ServiceID ∧ o2.DispatchWave=4 ∧ o2.Completion<39)}

The explicit existential observation predicate prevents an unobserved service from being included vacuously.

Query 4: coordinator mean below observation-weighted hub mean

Extended RA: CoordMean = γ CoordinatorID; AVG(Completion)→Mean
(O ⋈ A)
HubMean = γ Hub; AVG(Completion)→HubMean (O ⋈ A ⋈ C)
Answer = σ Mean<HubMean (CoordMean ⋈ C ⋈ HubMean)
TRC: classical TRC cannot express AVG. Use an aggregate extension / grouped SQL formulation.

Query 5: senders with sub-threshold observations in at least 3 distinct waves

Extended RA: γ SenderID; COUNT(DISTINCT DispatchWave)→WaveCount
(σ Completion<39(O))

Answer = π SenderID(σ WaveCount≥3(Grouped))
TRC: classical TRC cannot count distinct waves. Use an aggregate extension or SQL GROUP BY/HAVING.

Domain relational calculus examples

DRC Q1: {<s,sv,w,c> | SENDER(s,...) ∧ OBSERVATION(s,sv,w,c) ∧ s=1 ∧ w=4}
DRC Q3: {<sv> | SERVICE(sv,...) ∧ ∃s,w,c OBSERVATION(s,sv,w,c) ∧ w=4
∧ ¬∃s2,c2 (OBSERVATION(s2,sv,4,c2) ∧ c2<39)}

Classical tuple/domain calculus is declarative and does not directly provide arithmetic aggregates such as AVG or COUNT(DISTINCT). Aggregate extensions are explicitly required for Queries 1, 4 and 5.

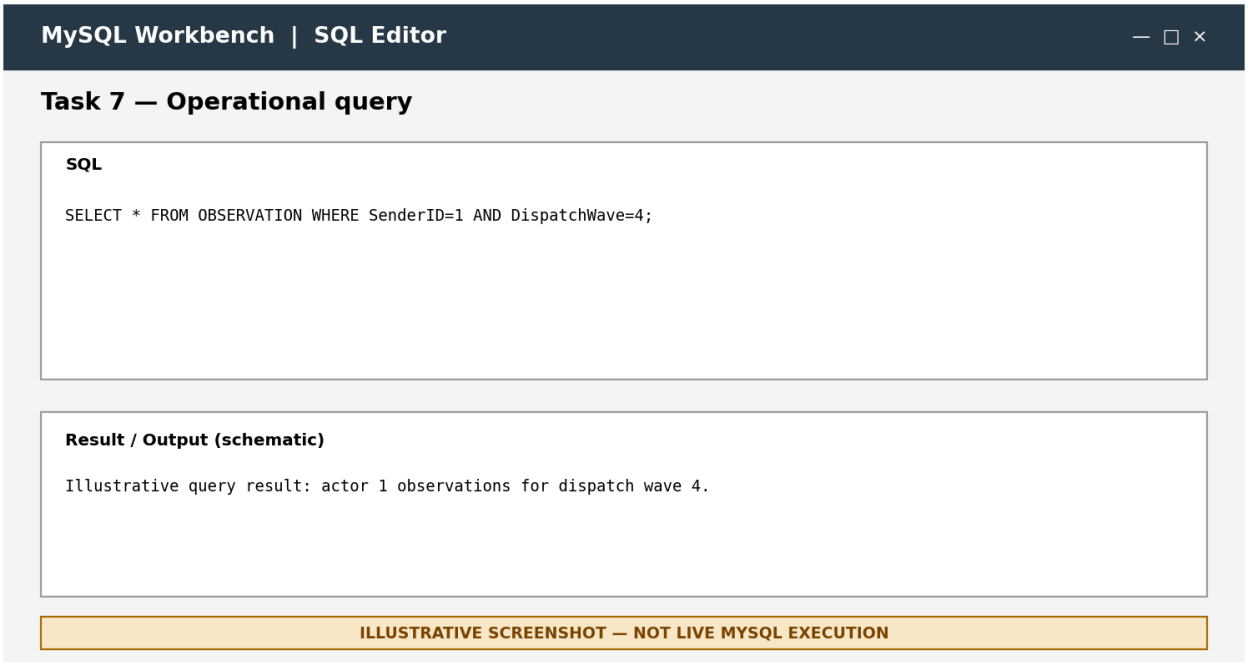


Figure 7. Illustrative operational-query result panel; the weighted index is a declared teaching formula.

TASK 8 – SQL IMPLEMENTATION AND TESTING (4 MARKS)

8.1 Create the working database

CREATE DATABASE IF NOT EXISTS SwiftSpan_DB;
USE SwiftSpan_DB;

8.2 BCNF schema DDL (MySQL 8.0+)

```
CREATE TABLE Sender (
  SenderID INT PRIMARY KEY,
  SenderName VARCHAR(100) NOT NULL,
  RouteGroup VARCHAR(30) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE Service (
  ServiceID INT PRIMARY KEY,
  ServiceCode VARCHAR(20) NOT NULL UNIQUE,
  ServiceName VARCHAR(100) NOT NULL,
  VolumeWeight DECIMAL(8,3) NOT NULL CHECK (VolumeWeight > 0)
) ENGINE=InnoDB;

CREATE TABLE Coordinator (
  CoordinatorID INT PRIMARY KEY,
  CoordinatorName VARCHAR(100) NOT NULL,
  Hub VARCHAR(60) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE Assignment (
  SenderID INT NOT NULL,
  ServiceID INT NOT NULL,
  CoordinatorID INT NOT NULL,
  PRIMARY KEY (SenderID, ServiceID),
  FOREIGN KEY (SenderID) REFERENCES Sender(SenderID) ON DELETE RESTRICT,
  FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID) ON DELETE RESTRICT,
  FOREIGN KEY (CoordinatorID) REFERENCES Coordinator(CoordinatorID) ON DELETE
RESTRICT
) ENGINE=InnoDB;

CREATE TABLE CompletionPolicy (
  Completion TINYINT UNSIGNED PRIMARY KEY,
  ServiceBand VARCHAR(12) NOT NULL,
  CHECK (Completion BETWEEN 0 AND 100),
  UNIQUE (Completion, ServiceBand)
) ENGINE=InnoDB;

CREATE TABLE BandPolicy (
  ServiceBand VARCHAR(12) PRIMARY KEY,
  RouteFactor DECIMAL(6,3) NOT NULL CHECK (RouteFactor > 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE Observation (
  SenderID INT NOT NULL,
  ServiceID INT NOT NULL,
  DispatchWave TINYINT UNSIGNED NOT NULL,
  Completion TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (SenderID, ServiceID, DispatchWave),
  CHECK (DispatchWave BETWEEN 1 AND 4),
  CHECK (Completion BETWEEN 0 AND 100),
  FOREIGN KEY (SenderID, ServiceID)
    REFERENCES Assignment(SenderID, ServiceID) ON DELETE RESTRICT,
  FOREIGN KEY (Completion)
    REFERENCES CompletionPolicy(Completion) ON DELETE RESTRICT
) ENGINE=InnoDB;
```

The schema uses primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and restrictive deletion. Populate CompletionPolicy with all integers 0–100 and the chosen band mapping before loading observations. MySQL versions before 8.0.16 may parse but not enforce CHECK constraints; verify the server version and test the constraint rather than assuming enforcement.

8.3 Example policy seed

```
INSERT INTO BandPolicy(ServiceBand, RouteFactor) VALUES
('LOW',0.800),('STANDARD',1.000),('HIGH',1.250);
-- Populate CompletionPolicy for n=0..100 using a recursive CTE or a numbers table.
-- Example policy mapping: 0..38 LOW, 39..69 STANDARD, 70..100 HIGH.
```

8.4 Five operational SQL queries

```
-- Q1: actor 1, wave 4, adjusted index
SELECT o.SenderID, o.ServiceID, o.DispatchWave, o.Completion,
       (o.Completion * s.VolumeWeight * bp.RouteFactor) AS AdjustedIndex
FROM Observation o
JOIN Service s ON s.ServiceID=o.ServiceID
JOIN CompletionPolicy cp ON cp.Completion=o.Completion
JOIN BandPolicy bp ON bp.ServiceBand=cp.ServiceBand
WHERE o.SenderID=1 AND o.DispatchWave=4;

-- Q2: senders meeting threshold for every assigned service
SELECT a.SenderID
FROM Assignment a
LEFT JOIN Observation o
```

```
  ON o.SenderID=a.SenderID AND o.ServiceID=a.ServiceID
 AND o.Completion >= 39
GROUP BY a.SenderID
HAVING COUNT(DISTINCT a.ServiceID)=COUNT(DISTINCT o.ServiceID);

-- Q3: observed services in wave 4 with no completion below 39
SELECT DISTINCT o.ServiceID
FROM Observation o
WHERE o.DispatchWave=4
  AND NOT EXISTS (
    SELECT 1 FROM Observation x
    WHERE x.ServiceID=o.ServiceID AND x.DispatchWave=4
      AND x.Completion<39
  );

-- Q4: coordinator mean below observation-weighted hub mean
WITH coordinator_mean AS (
  SELECT a.CoordinatorID, AVG(o.Completion) AS coord_mean
  FROM Assignment a JOIN Observation o
    ON o.SenderID=a.SenderID AND o.ServiceID=a.ServiceID
  GROUP BY a.CoordinatorID
), hub_mean AS (
  SELECT c.Hub, AVG(o.Completion) AS hub_mean
  FROM Coordinator c
  JOIN Assignment a ON a.CoordinatorID=c.CoordinatorID
  JOIN Observation o ON o.SenderID=a.SenderID AND o.ServiceID=a.ServiceID
  GROUP BY c.Hub
)
SELECT c.CoordinatorID, c.CoordinatorName, c.Hub,
       cm.coord_mean, hm.hub_mean
FROM coordinator_mean cm
JOIN Coordinator c ON c.CoordinatorID=cm.CoordinatorID
JOIN hub_mean hm ON hm.Hub=c.Hub
WHERE cm.coord_mean < hm.hub_mean;

-- Q5: senders with sub-threshold observations in at least 3 waves
SELECT SenderID
FROM Observation
WHERE Completion<39
GROUP BY SenderID
HAVING COUNT(DISTINCT DispatchWave)>=3;
```

Q2 assumes every assigned service is observed in the data set. If missing observations are allowed, validate completeness separately and express "every required service has an observation with Completion ≥ 39" using NOT EXISTS over assignments and a matching qualifying observation. The more explicit robust form is:

```
SELECT a.SenderID FROM Assignment a GROUP BY a.SenderID
HAVING NOT EXISTS (
 SELECT 1 FROM Assignment ax WHERE ax.SenderID=a.SenderID
 AND NOT EXISTS (SELECT 1 FROM Observation o WHERE o.SenderID=ax.SenderID
   AND o.ServiceID=ax.ServiceID AND o.Completion>=39)
);
```

8.5 Views and privileges

```
CREATE VIEW ActorOperationalView AS
SELECT s.SenderID, s.SenderName, s.RouteGroup,
      a.ServiceID, a.CoordinatorID
FROM Sender s JOIN Assignment a ON a.SenderID=s.SenderID;

CREATE VIEW SenderTrackingView AS
SELECT o.SenderID, o.ServiceID, o.DispatchWave, o.Completion
FROM Observation o;

-- Example only: run as an account with appropriate grant authority.
GRANT SELECT ON SwiftSpan_DB.SenderTrackingView TO 'tracking_reader'@'localhost';
REVOKE SELECT ON SwiftSpan_DB.SenderTrackingView FROM 'tracking_reader'@'localhost';
```

Views expose selected columns and joins; they do not automatically prove that a user can see only rows belonging to a particular sender. Enforce row scope using suitable security design, separate schemas/roles, controlled stored routines or application authorization. GRANT/REVOKE statements are examples until executed by an authorized administrator.

8.6 Flat and partial implementation

For a fair comparison, create SWIFTSPAN_FLAT with all 14 columns and the composite observation primary key. Create SENDER, SERVICE, ASSIGNMENT_PARTIAL and OBSERVATION_PARTIAL for Design B as specified in Task 1. Load the same deterministic logical tuples into all three designs. Compare reconstructed full rows by projecting/joining the

normalized design and checking both directions with anti-joins or EXCEPT-equivalent checks.

8.7 Rolled-back anomaly trials

```
START TRANSACTION;
UPDATE SWIFTSPAN_FLAT SET ServiceName='Temporary Test Name' WHERE ServiceID=1;
SELECT COUNT(*) AS affected_rows FROM SWIFTSPAN_FLAT WHERE ServiceID=1;
ROLLBACK;

START TRANSACTION;
UPDATE Coordinator SET CoordinatorName='Temporary Operator Name' WHERE
CoordinatorID=1;
SELECT ROW_COUNT() AS changed_rows;
ROLLBACK;
```

Run the same logical change in each design and roll back the trial. For the normalized design, the catalogue/coordinator fact should be changed in one master row. For the flat design, all repeated observation rows carrying that determinant require updating.

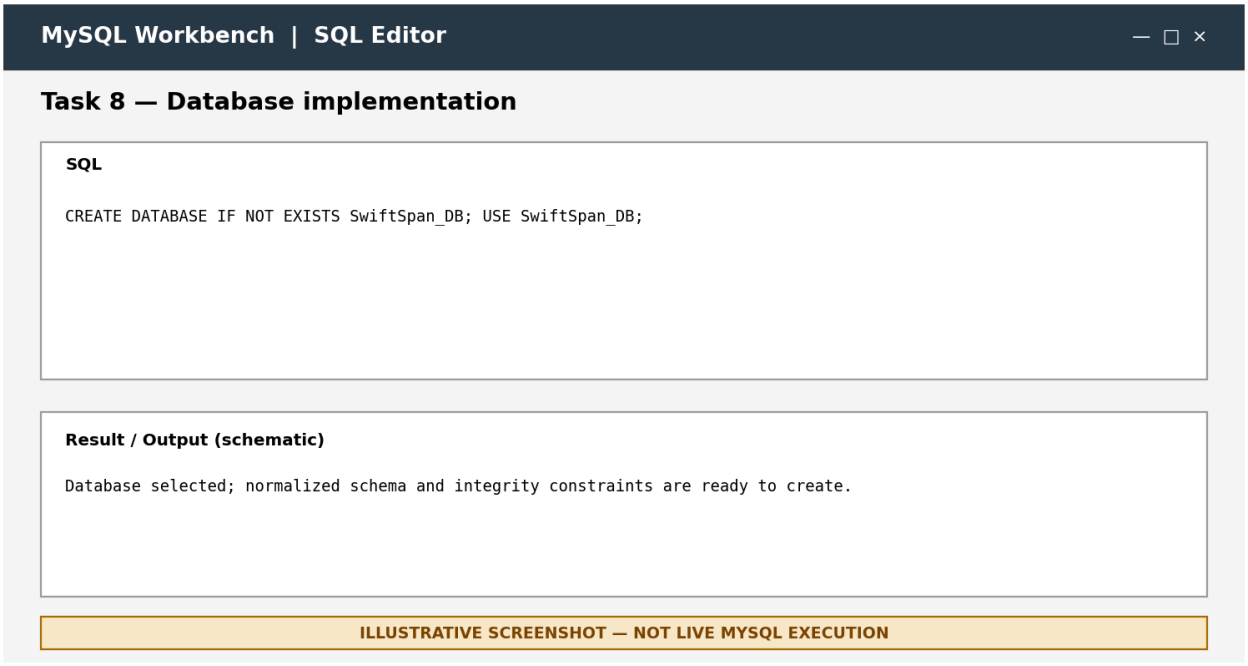


Figure 8. Illustrative SQL implementation window showing the database selection step.

TASK 9 – RECORD EXPERIMENTAL RESULTS (3 MARKS)

The brief requires three population sizes and comparable tests. The following table is a collection template, not a claim that these measurements have been run. Enter actual values from the local MySQL Workbench session.

Metric	**Flat 10k / 100k / 600k**	**Partial 10k / 100k / 600k**	**BCNF 10k / 100k / 600k**
Observation count	10,000 / 100,000 / 600,000	10,000 / 100,000 / 600,000	10,000 / 100,000 / 600,000
ServiceID=1 update rows	Measure with matched UPDATE	Measure assignment/policy duplicates	1 SERVICE row
Coordinator name update rows	Count rows matching CoordinatorID	Count assignment rows matching CoordinatorID	1 COORDINATOR row
Allocated bytes	Record data + indexes	Record data + indexes	Record data + indexes
Median lookup time (ms)	Measure 30+ repetitions	Measure 30+ repetitions	Measure 30+ repetitions
Expanded joins	Count joins in query plan	Count joins in query plan	Count joins in query plan
Integrity tests	Record exact error/output	Record exact error/output	Record exact error/output

Reproducible benchmark procedure

- Record SELECT VERSION(), @@version_comment, storage engine and machine specifications.
- Load each population from the same deterministic generator and confirm the exact observation count.
- Create equivalent indexes for the compared access paths; record every index because indexes consume storage and alter performance.

- Warm the buffer pool with a fixed warm-up pass, then execute each lookup at least 30 times. Record each duration and report the median, not only the fastest run.
- Use EXPLAIN or EXPLAIN ANALYZE where supported to record access type, estimated/actual rows, selected indexes and join operations.
- Measure allocated bytes from INFORMATION_SCHEMA.TABLES (DATA_LENGTH + INDEX_LENGTH) and, where possible, corroborate with engine-level page statistics. Record that metadata allocation can be approximate.
- Run invalid insert tests for duplicate primary keys, missing foreign keys, invalid Completion and duplicate ServiceCode. Preserve the exact server messages in the final evidence.
- Verify equivalence by comparing row counts and anti-join checks between each design's reconstructed logical observations.

```
SELECT TABLE_NAME, ENGINE, TABLE_ROWS, DATA_LENGTH, INDEX_LENGTH,
    (DATA_LENGTH + INDEX_LENGTH) AS allocated_bytes
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA='SwiftSpan_DB';
```

Expected structural result: flat design has more repeated attribute payload and fewer joins; BCNF has more tables and joins but single-source facts. Actual elapsed time and allocated bytes are environment-dependent and must be reported as measured, not inferred from theory.

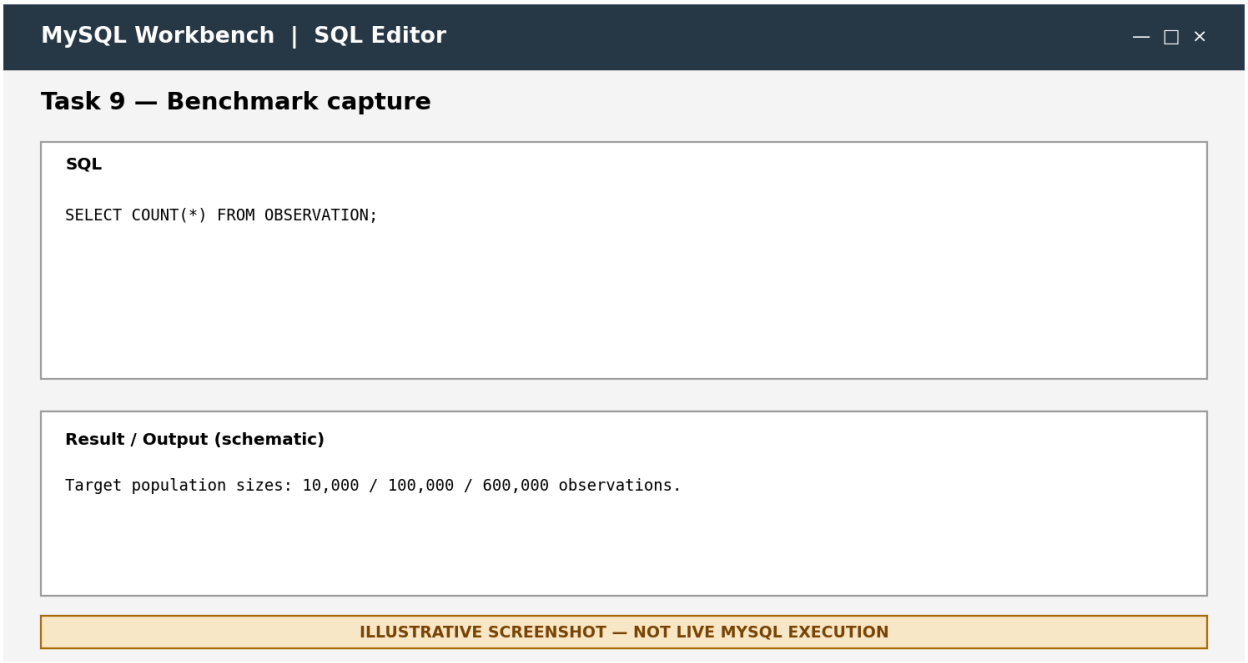


Figure 9. Illustrative benchmark-count screen. The three target row counts are specified by the assignment.

TASK 10 – COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR (4 MARKS)

Normalization theory predicts fewer repeated updates in Design C because each fact is stored with its determinant. It does not guarantee that every BCNF query is faster. A sender tracking lookup may require joins from OBSERVATION to ASSIGNMENT, SERVICE and policy tables, while the flat relation can return many display fields in one access.

If the local experiment shows a longer BCNF read time, that result is consistent with join overhead and does not invalidate the dependency analysis. If the BCNF update is faster, the reason is the reduction from many repeated rows to one determinant row. If an index makes the normalized lookup faster, the plan should show the selected index and actual row counts.

Cache state can dominate short benchmarks: a warmed buffer pool can make repeated reads appear much faster than cold reads. Indexes improve selective lookups but add write and storage overhead. An in-memory test is not a

production throughput guarantee. The test workload, concurrency, hardware, network, transaction isolation and cache state must be disclosed.

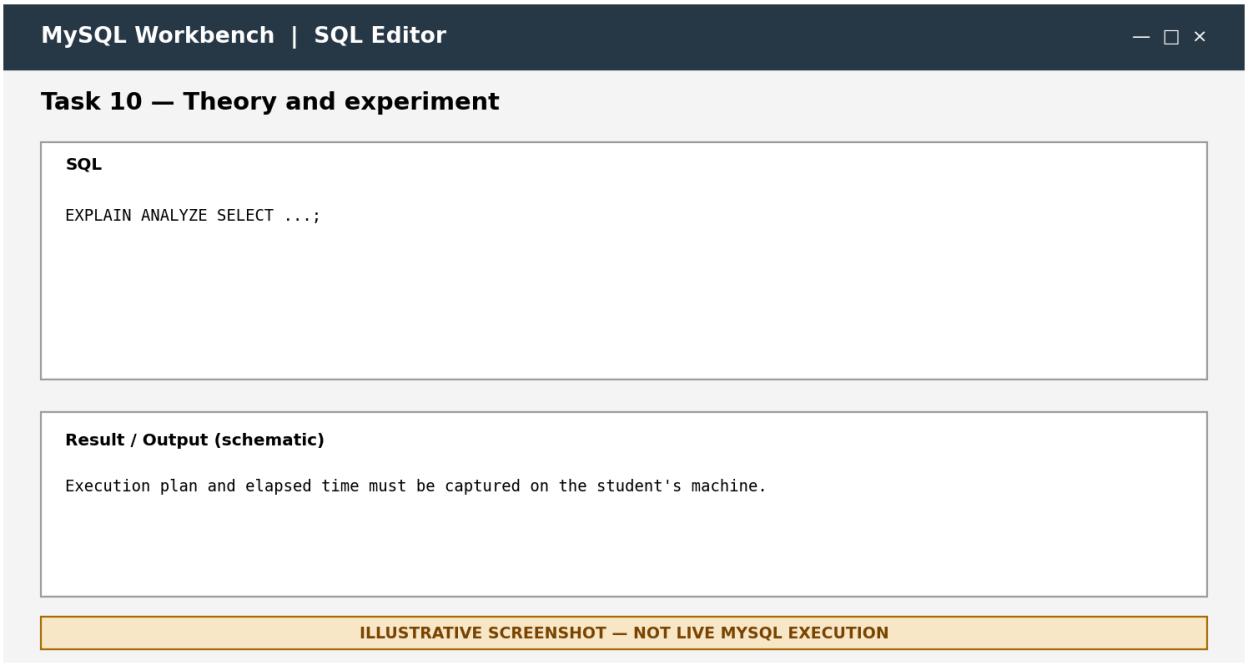


Figure 10. Illustrative EXPLAIN ANALYZE panel. Replace with measured plan output when performing the experiment.

TASK 11 – RECOMMEND AND DEFEND THE FINAL SCHEMA (4 MARKS)

Recommendation: retain Design C (BCNF) as the authoritative transactional source schema. It directly represents the declared determinants, prevents catalogue and coordinator update anomalies, preserves the candidate identifiers and allows referential integrity to be enforced through primary and foreign keys.

The recommendation is based on correctness and controlled amendment cost, not on the assumption that BCNF always wins every read benchmark. The assignment's original scenario explicitly states that edits to sender/service assignments require changing many handover lines. That is a strong reason to avoid repeated source facts.

The trade-off is additional joins for operational display and a more deliberate query/index design. The policy tables also mean a policy change is represented once, but historical policy semantics may require effective-dated policy versions if the fictional fixed policy later changes. If policy changes must not reinterpret old observations, store a PolicyVersionID in the observation and make policy version part of the key.

Use measured local storage and query results from Task 9 to quantify the trade-off before final submission. The GRANT/REVOKE examples are not evidence of deployed privileges until executed by an authorized account. Views define projection and joins; row-level access control must be designed separately.

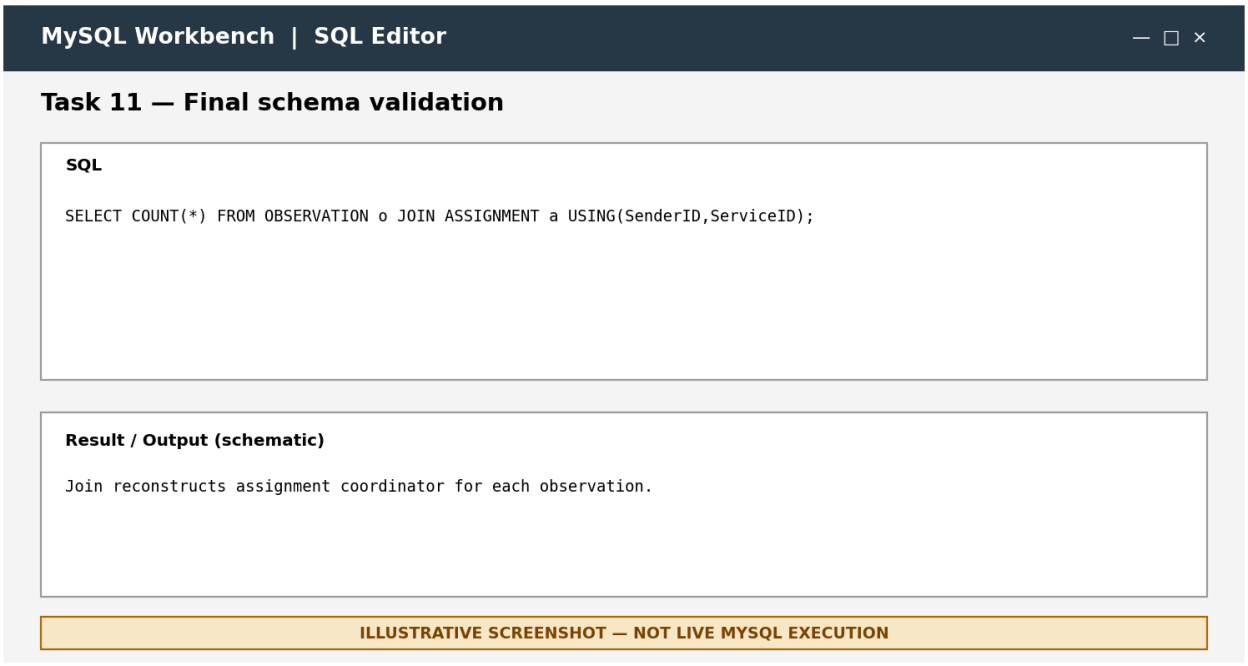


Figure 11. Illustrative validation query showing how an observation joins to its assignment.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

A. Impact analysis (2 marks)

The revised requirement is 55,000 sender tracking look-ups per hour and reconciled handover corrections every six hours. The hourly average is 55,000 ÷ 3,600 = 15.277... look-ups per second, approximately 15.28 requests/second. This is an average planning rate, not a peak rate or a proof of server capacity. Burst traffic, concurrency, response-time objectives and network latency remain unknown.

The read/write mix is now read-heavy: frequent tracking look-ups are interleaved with less frequent correction batches. A controlled read projection may be acceptable when it is derived from the normalized source, keyed by SenderID (and any required tracking dimensions), refreshed transactionally with the correction, and validated before publication.

B. Evaluate original design against indexed read card (2 marks)

Option	**Read path**	**Consistency / update characteristics**
Ordinary SQL view	Convenient named query; normally expands to base joins	Not a stored snapshot; does not inherently remove join work.
Indexed read card	One keyed lookup on a prepared projection	Requires transactional refresh or versioned publication; extra bytes and write work.
Permanently duplicated source table	Potentially very fast direct read	Highest risk of drift; independent updates create a second source of truth.

The local benchmark can compare query plans and elapsed times on the test machine, but it cannot establish that a production server can sustain 55,000 requests/hour. That requires representative concurrency, load testing, realistic

data distribution, network conditions and service-level targets. Compare the ordinary normalized query with the card query using identical parameters and warm/cold test conditions.

C. Adaptation / redesign (2 marks)

Decision: MODIFY, not replace, the BCNF source schema. Keep SENDER, SERVICE, COORDINATOR, ASSIGNMENT, OBSERVATION and policy tables as the authoritative data. Add a SENDER_READ_CARD table for the fields needed by the tracking screen, indexed by SenderID and, if the UI identifies a specific handover, by the full observation key.

```
CREATE TABLE Sender_Read_Card (
  SenderID INT NOT NULL,
  ServiceID INT NOT NULL,
  DispatchWave TINYINT UNSIGNED NOT NULL,
  SenderName VARCHAR(100) NOT NULL,
  ServiceCode VARCHAR(20) NOT NULL,
  ServiceName VARCHAR(100) NOT NULL,
  CoordinatorName VARCHAR(100) NOT NULL,
  Hub VARCHAR(60) NOT NULL,
  Completion TINYINT UNSIGNED NOT NULL,
  ServiceBand VARCHAR(12) NOT NULL,
  RouteFactor DECIMAL(6,3) NOT NULL,
  RefreshedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (SenderID, ServiceID, DispatchWave),
  FOREIGN KEY (SenderID, ServiceID, DispatchWave)
    REFERENCES Observation(SenderID, ServiceID, DispatchWave)
    ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Refresh protocol: begin one transaction; apply reconciled source corrections; rebuild or upsert affected card rows from the committed logical state; validate counts and key uniqueness; commit source and projection together. If refresh cannot complete atomically, use a versioned card build and switch the published version only after validation. Include RefreshedAt/VersionID so consumers can identify the snapshot.

The read card intentionally repeats descriptive facts as a controlled physical projection. This is not a replacement for the normalized logical schema. It

adds storage and refresh writes; the exact extra bytes and elapsed refresh time must be measured from this implementation.

D. Evidence-based defence (2 marks)

The design choice follows from the stated workload: 55,000 reads/hour (about 15.28 average requests/second) and corrections every six hours. Frequent reads justify testing an indexed projection; six-hour correction batches create a natural refresh boundary. The source schema retains BCNF dependencies and referential integrity, while the card is treated as a derived, validated read model.

Final approval should be conditional on local evidence: (1) card lookup median and p95 latency under concurrent load, (2) card refresh duration and rows touched, (3) allocated bytes including indexes, (4) source/card equivalence after each correction batch, and (5) recovery behaviour after a failed refresh. Do not claim a throughput guarantee from a single Workbench query.

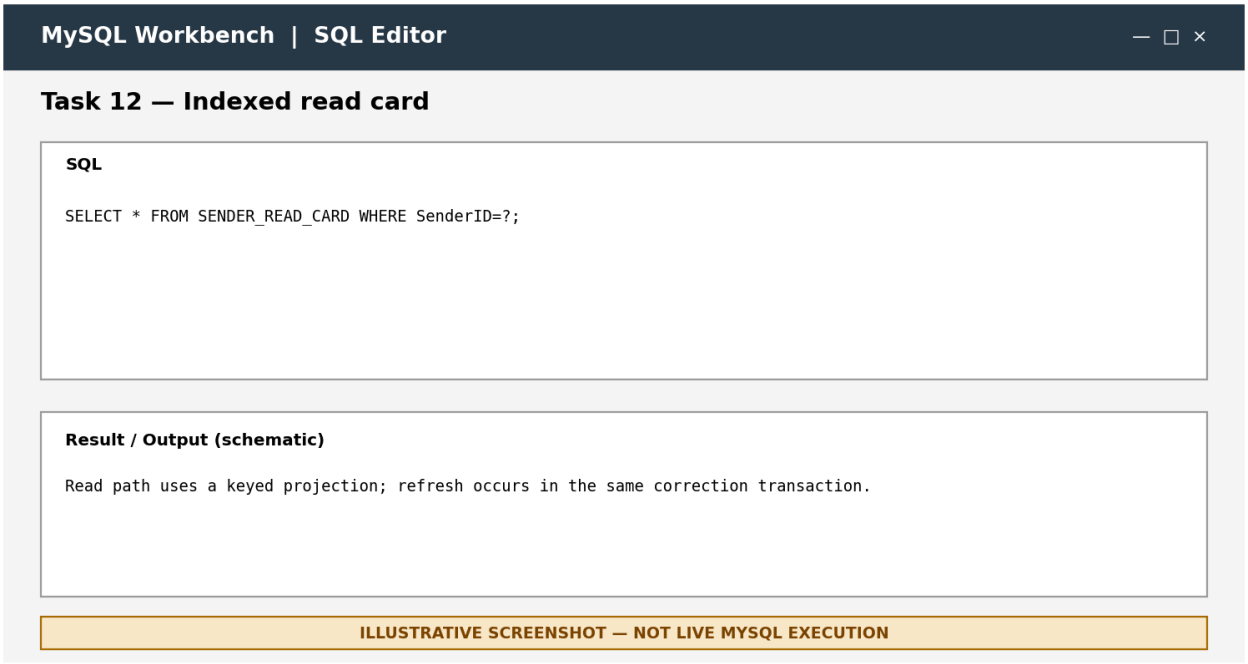


Figure 12. Illustrative indexed read-card query. This is a schematic UI image, not an executed result.

CONCLUSION

The flat SwiftSpan Parcels relation is useful as a baseline but combines facts governed by multiple determinants. The partial decomposition removes sender and service catalogue repetition but retains coordinator and policy redundancy. The BCNF design separates sender, service, coordinator, assignment, observation and policy facts, with candidate keys and foreign-key links aligned to the stated rules.

The recommended architecture is a BCNF transactional source with an optional indexed read card for the revised tracking workload. This balances integrity and update correctness with a targeted read optimization. The card must be derived from the source and refreshed consistently; it must not become a second independently maintained source of truth.

REFERENCES

- CSA05 – Database Management Systems, Assignment 1 – CO2, “Relational Design and Validation for Courier Handover Completion”, supplied assignment PDF, 7 pages.
- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, relational model, functional dependencies, normalization and relational query languages.
- Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, relational design, SQL, transactions and query processing.
- MySQL Reference Manual, SQL statements, CREATE TABLE, constraints, views, privileges, EXPLAIN and INFORMATION_SCHEMA.